

# Project 1: Spiking Wavefront Propagation

## Answer Sheet

ECE 613/T480: Neuromorphic Computing · Drexel University

### Group Information

*This project is completed in groups. List every member below (up to 5). Only one submission per group is needed.*

| Member    | Full name            | Drexel ID (e.g., abc123) |
|-----------|----------------------|--------------------------|
| Student 1 | Christopher Rui Zhao | cz438                    |
| Student 2 | Josh Bronner         | jb4595                   |
| Student 3 | Ilysia Krzywonos     | ik395                    |
| Student 4 |                      |                          |
| Student 5 |                      |                          |

**Date:** 8/2/26

**Notebook submission.** Upload your group's completed Colab notebook as a .ipynb file **to the course submission folder**, together with this answer sheet. Make sure **every cell has been run top to bottom** and that all outputs (spike table, visualizations, extracted path, systems measurements, and the Dijkstra comparison) are visible in the uploaded file.

*One submission per group.*

- ☐ We have uploaded our .ipynb notebook to the submission folder, with all cells run and outputs visible.
- ☐ The notebook runs top to bottom without errors.
- ☐ All group members listed above contributed to this work.

## Task 2: Wave Verification on the Uniform 4x4 Grid

Setup: 4x4 grid, uniform delays  $D_{ij} = 10$ , start (1,1), goal (4,4).

### 2a. Did the wavefront reach the goal?

Goal neuron (4,4) spiked: ☒ Yes ☐ No

First spike time of the goal,  $t_g$ : 60

Total number of spikes logged: 36

### 2b. Spike table (excerpt)

**Note: We used starting time  $t = 0$  instead of 1 to make clearer and cleaner outputs**

Paste an excerpt of your spike table. Include the first few timesteps and the timestep at which the goal fires.

Time: 0 Neuron: (1, 1)

Time: 10 Neuron: (1, 2)

Time: 10 Neuron: (2, 1)

Time: 20 Neuron: (1, 1)

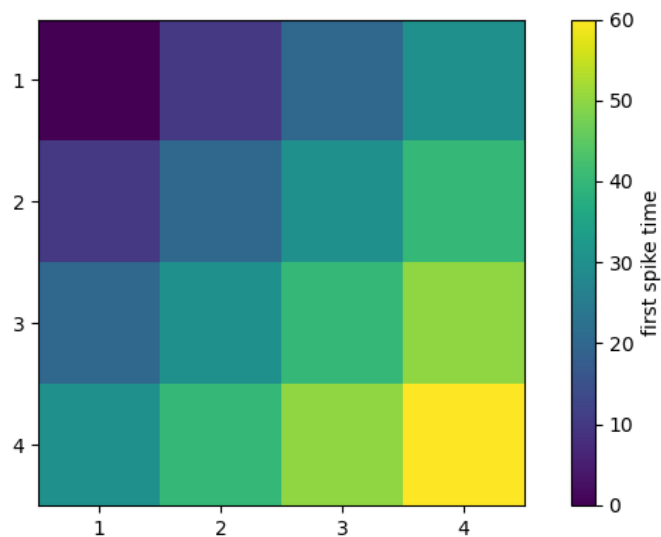
Time: 20 Neuron: (1, 3)

...

Time: 60 Neuron: (4, 4)

### 2c. Wave visualization

Paste your 4x4 first-spike-time heat map here. On a uniform grid you should see contours spreading outward from (1,1).



## Task 3: Path Extraction

---

### 3a. Extracted path

Write the path as an ordered list of grid coordinates, from (1,1) to (4,4).

[(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (4, 4)]

### 3b. Verification checks

- ☒ The path begins at (1,1) and ends at (4,4).
- ☒ Every consecutive pair in the path is a 4-neighbor (no diagonal or skipped steps).
- ☒ The path length equals the Manhattan distance.

**Path length (number of steps): 6**

**Manhattan distance for (1,1) → (4,4): 6**

**Do they match?** ☒ Yes ☐ No

*Note: on a uniform grid the optimal path is not unique. Any monotone staircase route from (1,1) to (4,4) is equally optimal, so your path may differ from another group's. Only the length must be minimal.*

## Task 4: The Systems View

### 4a. Cost of one plan (4×4, D = 10, (1,1) → (4,4))

| Quantity                                       | Your value |
|------------------------------------------------|------------|
| Total spikes                                   | 36         |
| SOPs ( $\approx 4 \times$ spikes)              | 144        |
| Convergence time $t_g$ (timesteps)             | 60         |
| Energy per plan ( $77 \text{ fJ} \times$ SOPs) | 11,088 fJ  |

### 4b. The delay-magnitude lever: D = 10 vs D = 5

Same grid, same endpoints, only the delay magnitude changes.

| D  | Total spikes | Convergence time $t_g$ | Extracted path                                           |
|----|--------------|------------------------|----------------------------------------------------------|
| 10 | 36           | 60                     | [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (4, 4)] |
| 5  | 36           | 30                     | [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (4, 4)] |

Did the path change? ☐ Yes ☒ No

Did the latency change? ☒ Yes ☐ No

What does this tell you about the absolute magnitude of the delays versus their relative values?

From my experiments, as long as the delays were greater than 2, we got the same convergence time and same total spikes. We got slightly lower spike counts with delays of only 2 or 1 due to the recovery time, but not by a lot. On the other hand, the relative value of the delays to each other is a big deal, since that is what causes the optimal path to potentially differ from any Manhattan traversal from start to goal.

#### 4c. Scaling with grid size (D = 10, corner to corner)

| Grid size | Total spikes | Convergence time $t_g$ |
|-----------|--------------|------------------------|
| 4 × 4     | 36           | 60                     |
| 8 × 8     | 272          | 140                    |
| 16 × 16   | 2112         | 300                    |

How does each quantity grow? Does spike count track the area of the grid or its width? Does convergence time track the same thing? Explain the difference.

Spike count seems to scale with grid width cubed: a doubling of grid width caused roughly an 8x increase in spike count from both 4 to 8 and 8 to 16. Spike count seems  $\approx \frac{1}{2} * \text{grid width cubed}$ .

Convergence time, on the other hand, seems to only roughly double with doubling of grid width.

This makes sense, since the convergence time should just be the time it takes for the wavefront to traverse from the top left to the bottom right of the grid, which is the width plus the height, roughly. On the other hand, spike count not only increases superlinearly as each neuron in the entire  $w \times h$  grid has to spike, but also even more than quadratically since spikes will also bounce backwards.

## Task 5: The 8×8 Roadblock Map

Setup: cost map `cost_map_8x8.csv` from the project folder. Open road = 1, mud = 25, roadblock = 1000. **Start (1,1), goal (8,8).**

### 5a. Your planned path

(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (2, 7), (3, 7), (3, 6), (3, 5), (3, 4), (3, 3), (3, 2), (4, 2), (5, 2), (6, 2), (6, 3), (6, 4), (6, 5), (6, 6), (6, 7), (6, 8), (7, 8), (8, 8)

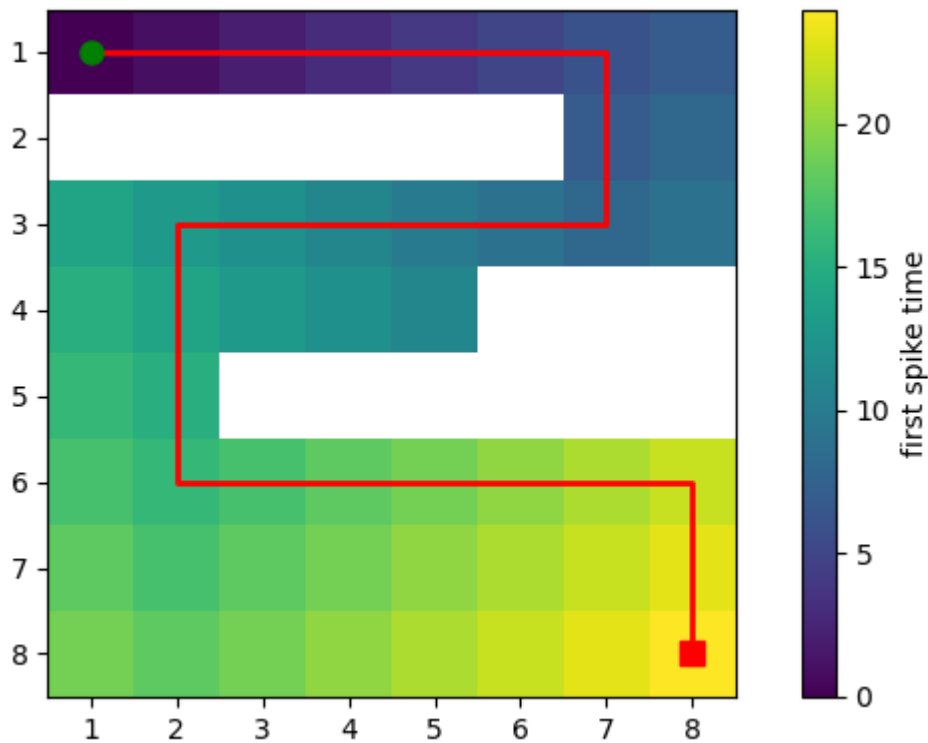
Total path cost: 24

Path length (cells): 25

Convergence time  $t_g$ : 24

Total spikes: 100

### 5b. Visualization: cost map with your path overlaid



### 5c. Verification against Dijkstra

|                     | Spiking wavefront | Dijkstra |
|---------------------|-------------------|----------|
| Total path cost     | 24                | 24       |
| Path length (cells) | 25                | 25       |

Do the total costs match? ☒ Yes ☐ No

*The optimal cost is unique, but the optimal path may not be. If two routes tie, the wavefront and Dijkstra may return different ones. Compare costs to decide correctness.*

### 5d. Interpretation

Does your path go through any roadblock (cost 1000)? Explain, in terms of axonal delays, why the wave routes around them, given that nothing in your code forbids entering them.

Our path did not go through any roadblocks. While nothing in our code explicitly forbids this, a roadblock of cost 1000 is 40x as much as the next most expensive terrain, that being mud of cost 25. While no part of the code explicitly forbids going through roadblocks, by the time the neuron spikes to traverse across one roadblock, a large majority of the rest of the grid can be expected to have fired, including the goal neuron.

On open road, the straight-line Manhattan cost from (1,1) to (8,8) would be 14. What did your path cost, and how much did the roadblocks add?

Extra cost = path cost - 14 = 24 - 14 = 10 (dijkstra's)

Does the path cross the mud (cost 25)? Why or why not?

No, the path does not ever travel through the mud. While the mud is far more efficient than a roadblock, for as long as there is a detour on the road, it is still far cheaper than going through mud. The cost to travel around a spot of mud is a few more than going straight, but it is far under the 25x cost of mud relative to simply moving to avoid it.

## Task 6: Write-Up

---

Answer each briefly, in your own words (2–4 sentences each).

### 6a. Why does the first spike of the goal neuron correspond to the optimal path, rather than any later spike?

Since each spike of a neuron corresponds to it being transversed, the first spike of the goal neuron represents the first time that it has been reached. The first spike represents the completion of the quickest way to the goal neuron, and any later spike represents a slower method, which is not optimal for the parameters that have been set.

### 6b. What is the role of the recovery variable $u$ ? What would go wrong with the wave if a neuron could fire again immediately?

The purpose of the recovery variable  $u$  is to avoid the same neuron spiking too often in short succession. Without it, the path could get stuck going backwards, or be caught in a loop between two neurons rather than traversing more difficult terrain. Having the recovery variable ensures that the path does not backtrack over the same locations.



**6c. You used a uniform delay of  $D = 10$ . What would change in the wave, and in the extracted path, if one region of the grid had a much larger delay?**

With a uniform delay, the neuron propagates spikes all at the same rate, and the wave would expand in all directions equally. If one region had a much larger delay, it would slow down the wave in that region and would move quickly through low-delay regions. The extracted path would move around that region and if there was no alternative then it would spike later.

**6d. This algorithm is event-driven and parallel: the whole wavefront advances at once, whereas a sequential graph search expands one node at a time. Name one advantage this gives on neuromorphic hardware.**

Speed. Through the event-driven and parallel architecture, this program can spike several neurons simultaneously to check multiple different paths at once, and stop as soon as the first way reaches the goal. As soon as the goal neuron has spiked once, the algorithm can conclude with certainty that the fastest way to the goal has been found. This allows the algorithm to complete the search much faster without losing any confidence, and allows it to save energy that would have been spent on any searches done after finding the fastest way from a traditional Dijkstra's algorithm.

**6e. From your Task 4 numbers: for this workload, is energy or latency the binding constraint, and which lever would you pull first to improve it?**

Energy was the binding constraint for us, since it scales cubically as the grid size increases, while latency only scales linearly. While latency does matter and our Python implementation sometimes took noticeable amounts of time for large grids or high delays, on real neuromorphic hardware, this time would be trivial. On the other hand, energy will become a constraint as we try to use this algorithm on real, practical maps that are much larger than  $8 \times 8$ . One way to improve it would be to just prevent cells from spiking more than once, since later spikes of the same cell cannot possibly be faster than earlier spikes and there is no reason for a path to ever go backwards on itself. This would at least constrain energy to grow at most quadratically.

Notes for the grader (leave blank)

---

| Component                                                             | Points | Earned |
|-----------------------------------------------------------------------|--------|--------|
| Task 1: Neuron model (Eqs. 1–4, genuinely spiking)                    | 20     |        |
| Task 2: Wave verification (reaches (4,4), spike table, visualization) | 15     |        |
| Task 3: Path extraction (back-tracing, checks pass)                   | 20     |        |
| Task 4: Systems view (costs, D lever, scaling)                        | 15     |        |
| Task 5: Roadblock map (path correct, matches Dijkstra cost)           | 20     |        |
| Task 6: Write-up                                                      | 10     |        |
| Total                                                                 | 100    |        |

Comments: